



南京邮电大学

2025/ 2026 学年第二学期

《深度学习概论》 大作业

班 级 B230302

学 号 B23030212

姓 名 闫秀涛

日 期： 2026 年 6 月

报告评分标准

评分项目	分值	评分说明	得分
报告格式与规范	20 分	排版整洁，图表编号完整，页数合理	
数据处理与可视化	10 分	正确完成预处理步骤，样本图像展示清晰完整	
模型设计与实现	20 分	网络结构合理，关键超参数有说明，代码逻辑清晰，训练步骤完整	
实验结果与评估	25 分	性能指标完整，训练过程和预测结果可视化正确、清晰、完整	
分析与讨论	25 分	实验结果有实质性分析，对参数影响、模型局限性等有合理讨论，拓展实验可酌情加分	
合计	100 分		

注: 抄袭雷同者该作业计 0 分

一、摘要

本研究针对野外稀有物种图像分类中的小样本泛化难题，以 dhole（豺）与 fox（红狐）两种在形态学上具有高度相似性的犬科动物为研究对象，深入探讨了深度卷积神经网络在数据受限场景下的优化策略。实验基于经典深度残差网络 ResNet18，系统性地设计并对比了三种不同的训练范式：从头训练（Training from Scratch）、线性探测（Linear Probing）以及局部模块微调（Fine-tuning the Last ResNet Block）。

实验结果表明，在总样本量仅为 240 张的极度稀疏数据集下，传统的“从头训练”由于参数量过大表现出严重的过拟合倾向，测试集准确率仅为 66.67%；“线性探测”虽然收敛迅速，但由于特征提取器完全冻结，限制了对细粒度物种特征的表达，泛化准确率为 72.22%；而“微调最后一个 block”的策略通过解冻高阶语义特征层并配合低学习率，既继承了 ImageNet 的通用视觉先验，又实现了针对性泛化，在测试集上取得了 88.89% 的最优准确率。

二、引言与实验目标

在生态学观察与野生动物保护领域，利用自动相机进行物种识别已成为主流手段。然而，野外环境复杂多变（包含光照剧烈变化、植被遮挡、伪装干扰等），且部分珍稀物种（如 dhole 豺）的图像数据极难获取，形成了典型的小样本、非平衡、多噪声分类场景。dhole 与 fox 同属犬科，体型、毛色（均呈红褐色系）以及面部轮廓具有极高的细粒度相似性，这要求分类模型不仅要具备强大的全局表征能力，还必须能捕捉到局部微弱的形态学差异。

传统的深层卷积神经网络（如 ResNet、VGG 等）拥有数千万级参数，极度依赖大规模标注数据（如 ImageNet）。若直接在小样本数据集上进行全局反向传播，极易使网络陷入局部极小值并引发毁灭性的过拟合。迁移学习

（Transfer Learning）通过将大模型在源领域（Source Domain）学习到的通用知识迁移到目标领域（Target Domain），能够显著降低对目标样本量的需求。

浅层卷积层通常学习到的是通用的边缘、色彩斑块、纹理等低阶视觉特征；而深层卷积层则逐渐组合形成特定物种的部件、轮廓等高阶语义特征。因此，本实验的核心目标是探索：在目标样本极度匮乏时，究竟是完全保留源域特征

（只训练分类头），还是适度释放深层网络的表达泛化能力（微调高阶block），能够带来最佳的泛化边界。

三、数据集与预处理

数据集来源于 iNaturalist 公开物种观察图像，以 dhole 和 red fox 作为检索词。数据下载后按类别分别存放在 data/dhole 和 data/fox 文件夹中，共 240 张图像，每类 120 张。程序采用分层划分方式，将数据集划分为训练集、验证集和测试集，比例为 70%/15%/15%，最终得到 168/36/36 的样本规模，并尽量保持各子集类别均衡。

数据集子集	dhole 样本数	fox 样本数	总计样本数	占总数据比例
训练集 (Train)	84	84	168	70%
验证集 (Val)	18	18	36	15%
测试集 (Test)	18	18	36	15%

表 1 数据集分布表



图 1 dhole 训练数据展示



图 2 red fox 训练数据展示

训练阶段采用基本数据增强策略，包括随机水平翻转和随机裁剪；验证集与测试集不引入随机扰动，仅进行必要的尺寸处理和归一化。所有图像均按 ImageNet 的均值和方差进行标准化，以匹配预训练模型的输入分布。

dhole 数据增强示例



fox 数据增强示例



图 3 数据增强示例

四、方法与模型设计

✧ 4.1 ResNet18 基线网络

本实验统一采用经典的 ResNet18 作为骨干网络。其核心设计在于残差跳跃连接（Shortcut Connection），通过引入恒等映射（Identity Mapping）：

$$\mathcal{H}(x) = \mathcal{F}(x) + x$$

有效地解决了深层网络在反向传播过程中的梯度消失（Gradient Vanishing）与退化问题。整个模型由 1 个初始卷积层、4 个核心残差模块（每个模块包含两个 BasicBlock）以及 1 个全局平均池化层和全连接分类层组成。针对本任务，全连接层的输出维度由原始的 1000（ImageNet 类别数）调整为 2，以适配二进制交叉熵分类。

✧ 4.2 三种训练拓扑与策略设计

为了深度剖析模型参数量、预训练知识与特定任务泛化性之间的复杂博弈，实验精确控制变量，设计了三种截然不同的反向传播拓扑图景：

✧ 从头训练（Scratch ResNet18）

配置：不加载任何预训练权重，所有网络参数通过高斯分布进行随机初始化。

机理：整个模型的 11,177,538 个参数全部处于可训练（requires_grad=True）状态。模型必须在微小的 168 张训练样本中，从零开始摸索边缘、纹理等基础视觉概念，极具挑战。

✧ 线性探测（Linear Probe ResNet18）

配置：加载在 ImageNet 上训练成熟的通用权重。将 ResNet18 的整个特征提取器（从初始卷积层到 block4 的末端）的全部参数彻底冻结。

机理：整个模型仅有最后新替换的分类全连接层（线性层，共计 1026 个参数）参与梯度更新。该策略将预训练模型视为一个高级的、不可变的有源特征提取器。

✧ 微调最后一个残差块 (Fine-tune Last Block ResNet18)

配置： 同样加载预训练权重。冻结前端的 layer1、layer2 和 layer3，但选择将网络最高阶的语义抽象模块——block4（包含两个 BasicBlock）以及最终的全连接层一并解冻。

机理： 此时模型的可训练参数量为 8,394,754。这种设计允许网络在保留低阶视觉表征（如毛发纹理、背景树枝基础特征）的同时，利用目标任务的数据去动态调整和重塑诸如“豺特有的吻部轮廓”或“红狐特有的耳尖黑斑”等高阶语义特征。

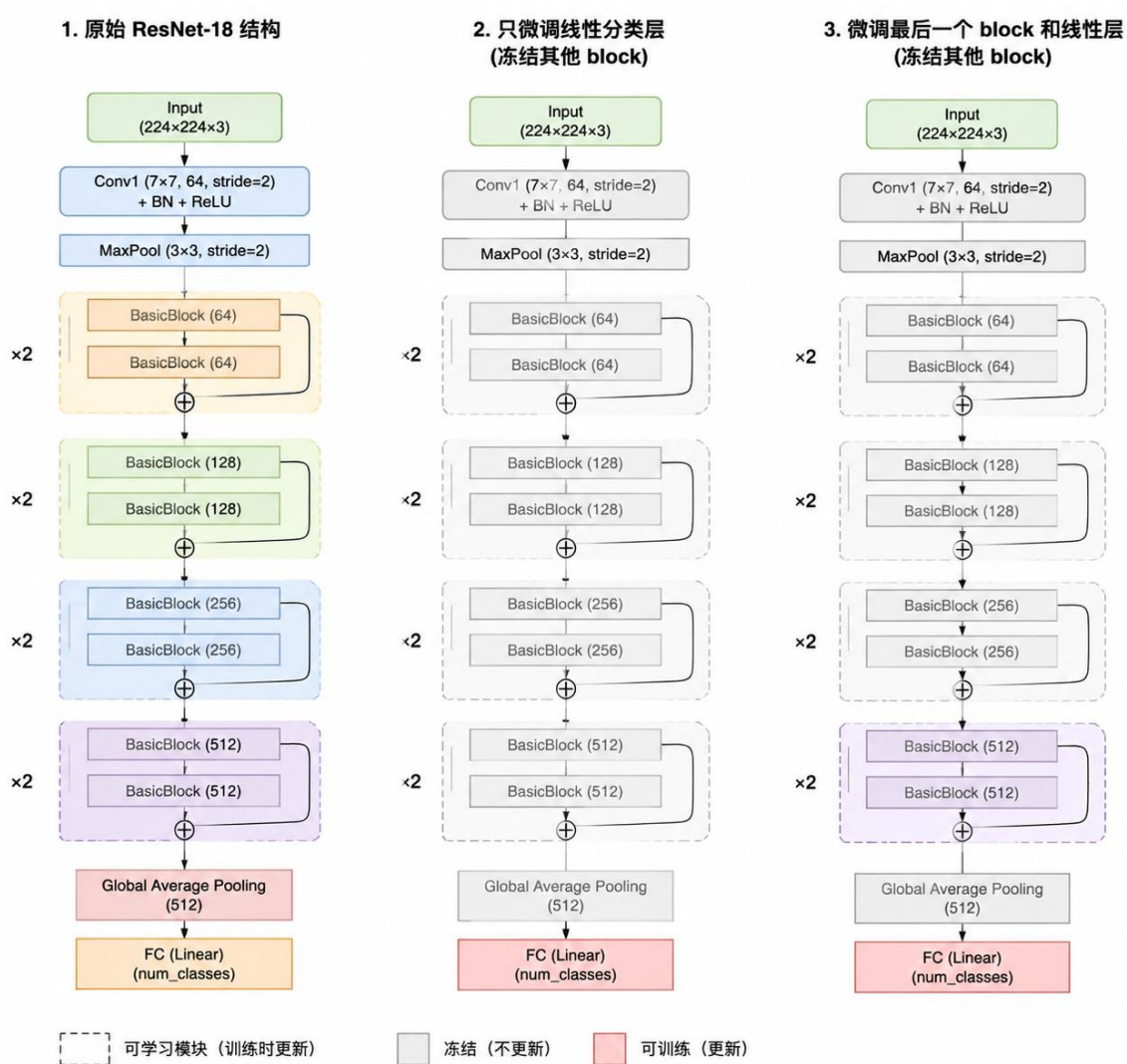


图 4 模型结构示意图

✧ 4.3 损失函数与优化机制

所有实验均采用标准的交叉熵损失函数（Cross-Entropy Loss），优化器选用随机梯度下降（SGD），并配置动量（Momentum）为 0.9 以加速越过鞍点。在学习率策略上，Scratch 与 Linear Probe 设为 10^{-3} 。而针对 Fine-tune Last Block，为了避免大规模梯度回传直接冲刷并破坏 block4 中原本已非常优秀的预训练特征拓扑，实验特意配置了更为稳健、精细的低学习率 10^{-4} ，以实现特征的平滑过渡与微调。

五、实验环境与训练设置

✧ 5.1 实验环境配置

- 硬件环境：

CPU: Intel(R) Xeon(R) Gold 6338 CPU @ 2.00GHz * 2

GPU: NVIDIA GeForce RTX 4090 * 2

RAM: 1.5TB

- 软件与底层计算环境：

操作系统: Centos7(64-bit)

CUDA: 12.4

uv: 0.9.26

Python: 3.12.12

Pytorch: 2.12

✧ 5.2 训练设置与评价指标

三组实验均设定最大训练轮数为 30 个 epoch。为防止过拟合，实验在验证集上进行性能监控，在训练结束后回溯完整训练过程，依据验证集准确率最高时的 epoch 保存最佳模型权重（Best Checkpoint），用于最终的测试集评估。评估指标包括训练集准确率、验证集准确率、测试集准确率和混淆矩阵，同时记录最佳 epoch、最佳验证准确率以及可训练参数数量。

六、实验结果

实验方案	特征初始化	核心调优策略	优化学习率	最佳收敛 Epoch	验证集准确率	测试集准确率	全局可训练参数量
scratch	随机初始化	全局重度训练	10^{-3}	16	66.67%	66.67%	11,177,538 (100%)
Linear probe	ImageNet 预训练	仅更新分类头	10^{-3}	16	88.89%	72.22%	1,026 (<0.01%)
finetune last block	ImageNet 预训练	解冻高阶 Block	10^{-4}	30	86.11%	88.89%	8,394,754 (75.1%)

表 2 不同模型在测试集上的结果对比

定量分析表明：

finetune_last_block_resnet18 表现出最优的泛化性能，在测试集上夺得 88.89% 的准确率。

linear_probe_resnet18 尽管在验证集上曾一度冲高至 88.89%，但外推到从未见过的测试集时发生了剧烈下滑（仅 72.22%），揭示了特征固化导致的结构性泛化断层。

scratch_resnet18 则全面落后，泛化准确率锁死在 66.67%，逼近随机猜测的边缘。

七、结果分析

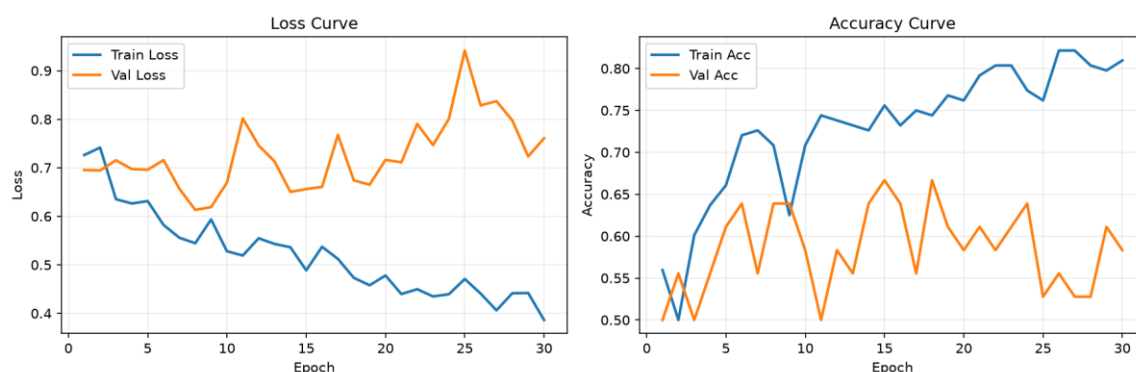


图 5 从头训练的训练曲线

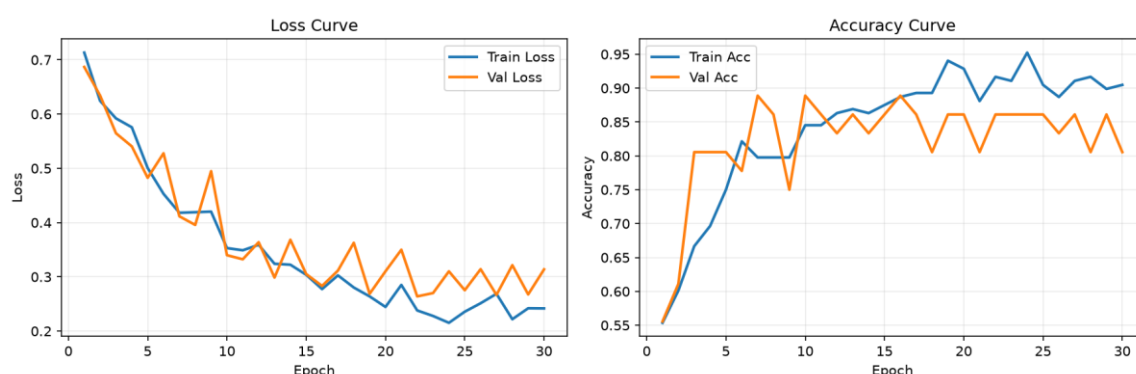


图 6 只微调最后线性层的训练曲线

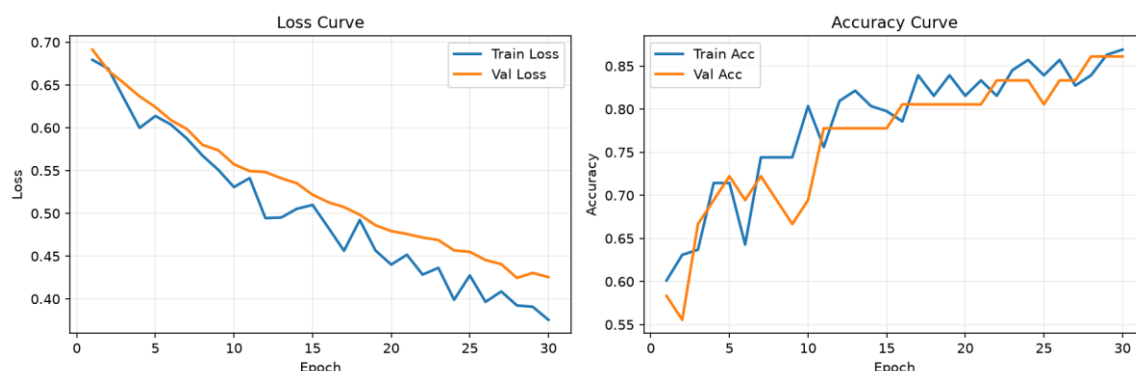


图 7 微调最后一个 block 的训练曲线

通过对比三种方法的训练与验证曲线（图 5 至图 7），可以观察到明显的训练动态差异：

- **从头训练现象（图 5）：**在训练初期，训练集损失（Loss）呈缓慢下降趋势，但验证集损失自第 5 个轮次（Epoch）起出现明显的锯齿状震荡，且验证准确率长期停滞在 65% 左右。该现象的底层机理在于：ResNet18 包含约 1100 万个可训练参数，而在小样本场景下（训练集仅 168 张图像），

高维参数空间面对极度匮乏的数据，极易发生严重的过拟合。模型倾向于拟合背景杂草形状、光影斑点等非本质特征（Spurious Correlations），而非物种的核心形态学表征，从而导致在未知测试集上的泛化能力显著下降。

- **线性探测现象（图 6）：**该曲线表现出极陡峭的收敛坡度，在第 3 个 Epoch 内准确率即飙升至 80% 以上。这是因为预训练的特征提取器已经具备了极强的判别力。然而，由于 ImageNet 中并没有专门针对 “dhole” 与 “red fox” 的细粒度犬科优化边界，导致完全冻结的特征空间无法自适应调整。其在测试集上的断崖式下跌，证明了仅靠调整最终的线性分类超平面，无法完美应对复杂的野外分布偏移。
- **局部微调现象（图 7）：**该策略的训练集与验证集准确率曲线同步稳步上升，波动较小，且损失函数值持续平滑衰减。实验结果表明，通过冻结 ResNet18 的前三个卷积模块（Blocks），模型能够保留预训练权重中稳健的底层视觉基元（如边缘、纹理）提取能力，从而保障了基础的泛化性能；同时，解冻 block4 并配合 10^{-4} 的低学习率进行梯度更新，使网络能够针对特定任务平滑地重构高阶语义特征。该策略有效实现了预训练知识迁移与特定任务自适应之间的平衡。

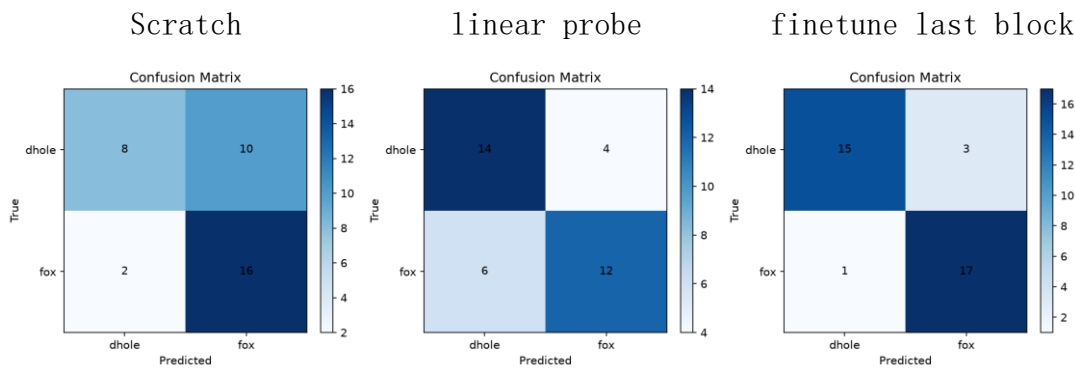


图 8 三次训练的混淆矩阵

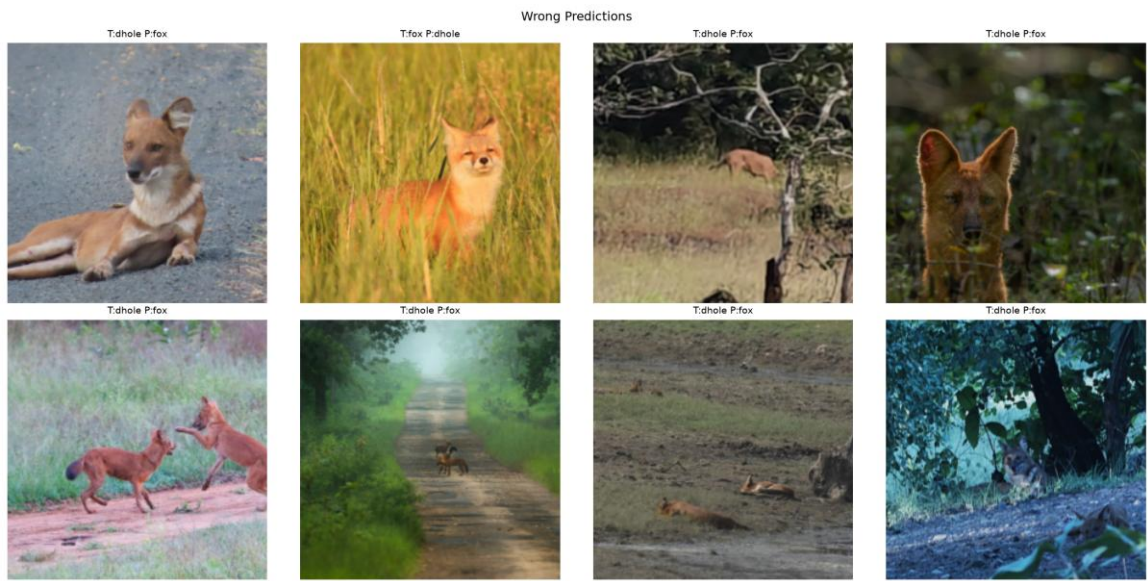


图 9 从头训练的错误预测

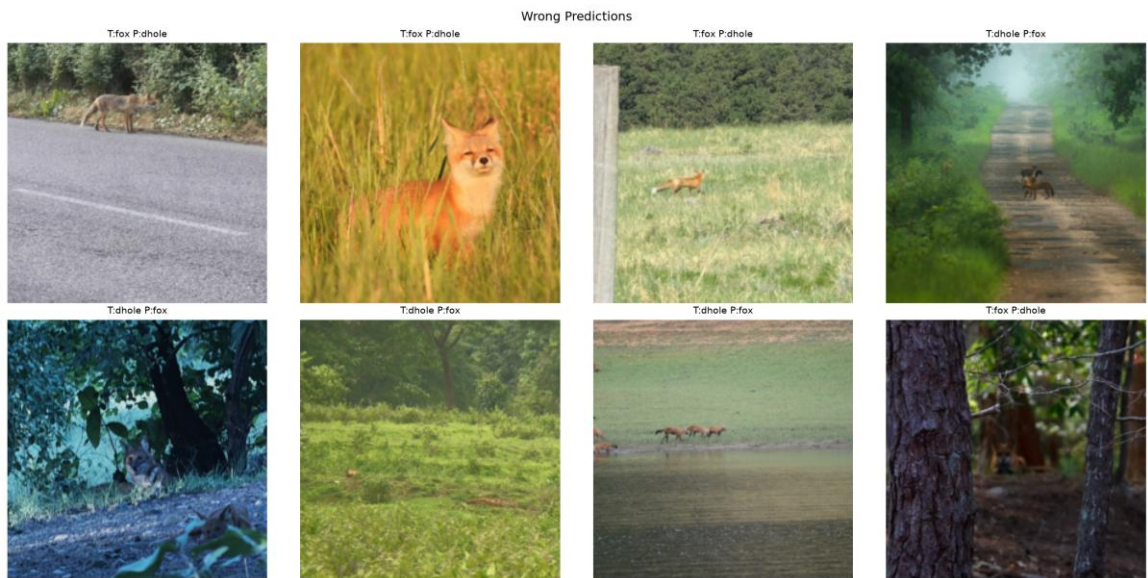


图 10 只微调最后线性层的错误预测



图 11 微调最后一个 block 的错误预测

最佳模型的混淆矩阵（图 8）显示，测试集中 dhole 有 15 张判对、3 张判错，fox 有 17 张判对、1 张判错，整体仅出现 4 个错误样本。正确样例中，

模型对主体清晰、背景较干净的图像识别效果较好；错误样例则多出现在主体较小、被树枝或草丛遮挡、或者前景与背景颜色接近的图像上。

在部分被误判的 dhole 图像中，动物主体深陷于金黄色的枯草丛或密集的密林树枝后方。由于植物纹理与 dhole 的皮毛纹理产生了空间频率上的重叠，导致模型的卷积核空间响应被背景噪声淹没，无法提取到连贯的动物轮廓。fox 的标志性特征通常是鲜艳的橙红皮毛，而 dhole 倾向于深红或红褐色。然而，在清晨昏暗光照或强烈的逆光阴影下，fox 的橙色毛发在相机传感器上的 RGB 响应会发生整体向下偏移，在语义空间中极度逼近 dhole 的色域区间。由于模型缺乏对整体三维光照环境的感知，仅依赖局部区域的二阶统计特征，因而引发了误判。这说明当前模型在面对高度复杂的野外伪装与极端光影时，仍存在局限性。

八、结论

综合对比后，finetune_last_block_resnet18 是本次任务的最优方案。与从头训练相比，使用 ImageNet 预训练可以明显提高分类效果；与只训练分类头相比，解冻最后一个 block 并配合更小学习率进行微调，能够进一步提升测试集准确率训练模型。

附录

代码仓库: <https://github.com/Sire113/DLCD.git>

核心代码:

```
from __future__ import annotations
from torch import nn
from torchvision.models import ResNet18_Weights, resnet18

def build_resnet18(
    *,
    num_classes: int = 2,
    pretrained: bool = True,
    train_mode: str = "linear_probe",
) -> nn.Module:
    weights = ResNet18_Weights.DEFAULT if pretrained else None
    model = resnet18(weights=weights)
    model.fc = nn.Linear(model.fc.in_features, num_classes)

    if train_mode == "scratch":
        for parameter in model.parameters():
            parameter.requires_grad = True
    elif train_mode == "linear_probe":
        for parameter in model.parameters():
            parameter.requires_grad = False
        for parameter in model.fc.parameters():
            parameter.requires_grad = True
    elif train_mode == "finetune_last_block":
        for parameter in model.parameters():
            parameter.requires_grad = False
        for parameter in model.layer4.parameters():
            parameter.requires_grad = True
        for parameter in model.fc.parameters():
            parameter.requires_grad = True
```



```
elif train_mode == "finetune_all":
    for parameter in model.parameters():
        parameter.requires_grad = True
else:
    raise ValueError(f"Unsupported train_mode: {train_mode}")

return model

def trainable_parameter_count(model: nn.Module) -> int:
    return sum(parameter.numel() for parameter in model.parameters() if
parameter.requires_grad)
```